

3.3 Terraform the whole stack

- When you finish it, your entire Cold Chain Log exists as reviewable code rather than as clicks nobody can reproduce.

Learning objective

[What I learnt from the video](#) (about terraform)

Read this ←←

How do resources reference each other in Terraform?

They reference each other through specific names given to them in terraform. I have made an note on this below:

NOTE AI: there are these on every resource that I have seen:

```
resource "aws_api_gateway_resource" "branches" {
```

Two quoted parts mean different things:

resource "TYPE_OF_RESOURCE" "TERRAFORM_NAME"

So this means:

- "aws_api_gateway_resource" = what AWS resource Terraform should create. In this case, a resource/path inside API Gateway, such as /branches .
- "branch" = your local Terraform name for it. Terraform uses this so you can refer to that resource elsewhere in your code.

A simple way to think of it is:

```
1 resource "what am I creating?" "what am I calling it in Terraform?"
```

So:

```
1 resource "aws_lambda_function" "coldchain_lambda"
```

Means: create. an AWS Lambda function, and inside my Terraform code I'll call it coldchain_lambda

LUKE EXPLANATION: there are these on every resource that I have seen:

```
resource "aws_api_gateway_resource" "branches" {
```

Luke just said that the whole thing is the name you gave it in terraform which is more believable because it does look like the part in a function and he also showed me why.

Why is hardcoding names and ARNs a problem?

DEF: An **ARN** (Amazon Resource Name) in AWS is a unique string that identifies a specific resource across the entire AWS ecosystem

The main thing is hardcoding makes the program less flexible, by introducing variables and outputs it makes the code less rigid and more flexible.

What is a data source, and when do you need one?

A data source in Terraform is a way to query and retrieve information from external systems, APIs, or existing infrastructure without creating or modifying any resources.

For example, I have used it to rap all the lambdas data together so when it is sent to AWS It is all used in the creation. I have used data resource beastie it gathers all the 'data' about the lambda.

What does Terraform do about the order things get created in?

Task

- i This section is a lot bigger and has taken me a lot longer than any other section in this week. When you finish it, the entire Cold Chain Log exists as reviewable code rather than as clicks nobody can reproduce.

We will be creating the whole project from the last two weeks, this includes:

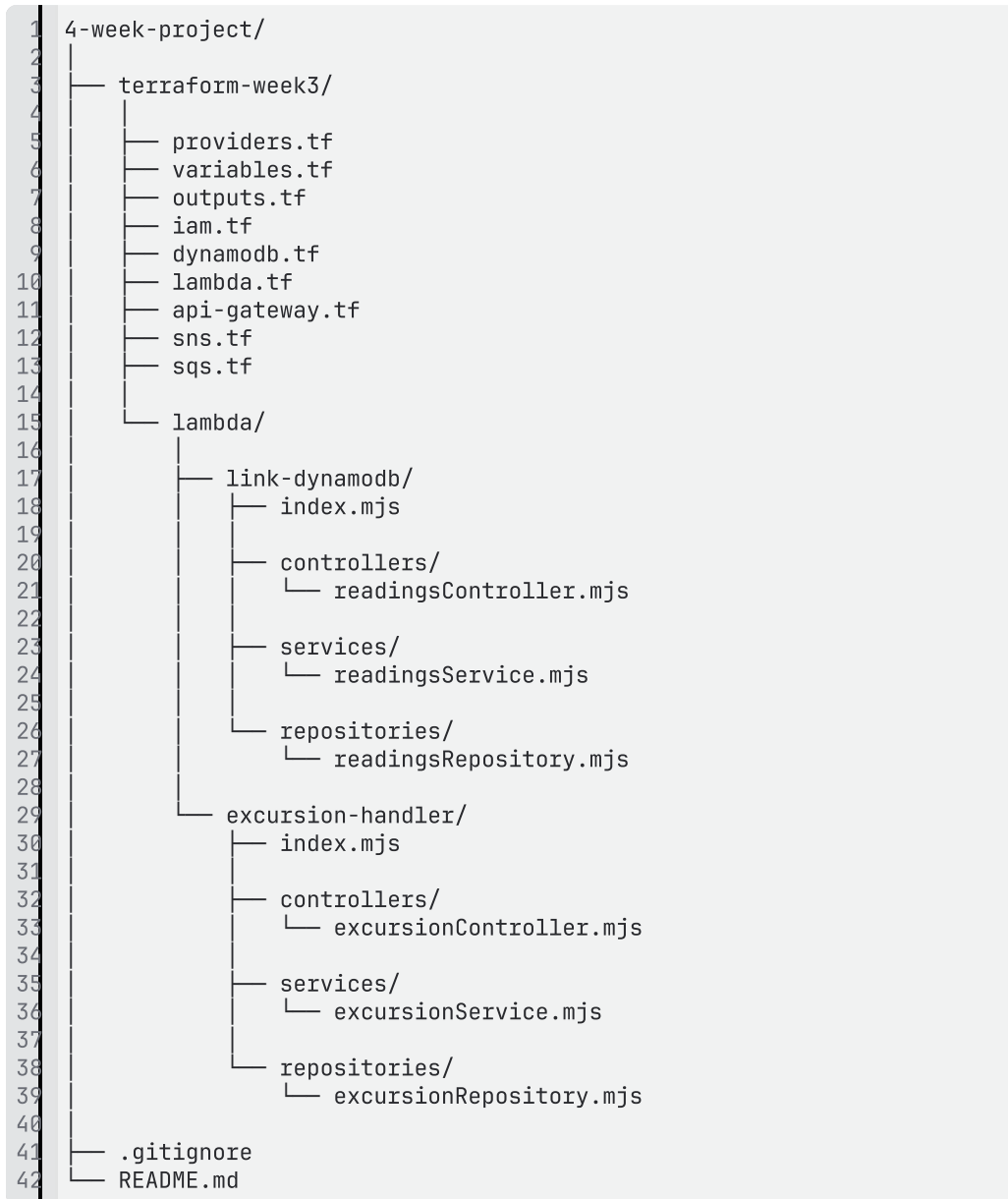
- The {{coldchain-readings}} DynamoDB table
- The create-reading Lambda and the excursion handler Lambda
- API Gateway with all three routes
- The SNS topic
- The SQS queue and its dead letter queue, and the subscription between topic and queue
- The event source mapping from the queue to the handler
- Every IAM role and permission tying them together — the create Lambda needs to write to Dynamo and publish to SNS; the handler needs to read the queue and update

FIRST: re-structing the files

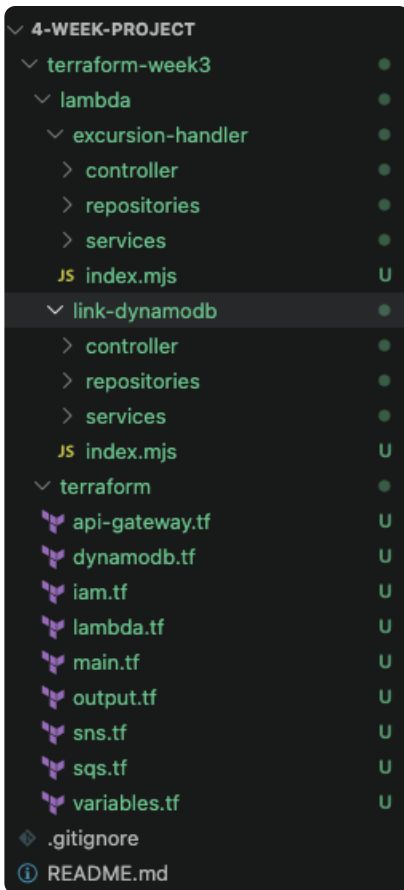
We are going to re structure the flies in my git repo first. we need to get ride off all the unnecessary file and we are going to structure the files around a basic from of Controller-Service-Repository structure that we learnt about in ticket 2.4.

New structure of files

This is the new structure of flies:



This is a visual representation of it in VScode



Understanding the new structure

READ THIS



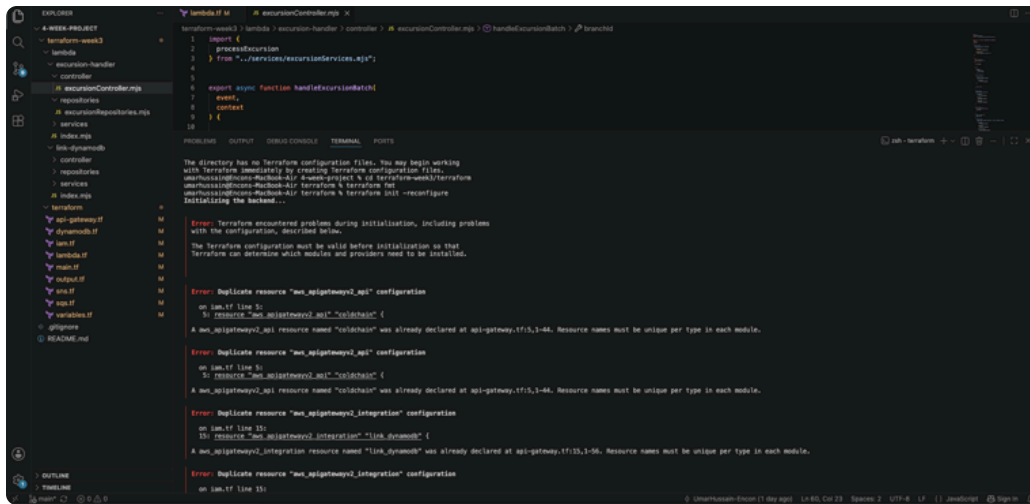
NOW: Making the whole project in terraform

In this section we will be doing the tickets task, I will be re-creating the whole project that was made in weeks 1-2 in terraform.

Stand up tomorrow: taking Rias words into consideration I changed the controller file code for my lambda to CASE statements rather than the AI's IF statements that were give to me.

This was done to optimise the code as the if state would have to run through each one before getting to the desired outcome they wanted. Whereas with Switch statements it would jump straight to it.

Even found picked up on an error with the terraform code that the AI had given me. the AI attemet to make the code clear by separating everything out (put more white space around the code) to make it easier for me and others to read. However, what this had done was make the code unusable. Hence the error you can see below



So we changed the code from this:

```
1 name =
2   var.api_name
```

To this:

```
1 name = var.api_name
```

- I have also changed the HTTPS APIs to RESTful APIs. ☒
- I have also added API keys (the security risk I was on about) ☒

Too do:

- UUIDv1() = currently the UUID is merged with the logs time and date stamp, I want it too be a sprat column and I want the 409 'you can not make a reading with the same date/time' to be made for the UUID instead. ☒

make sure it is separated from the date and time stamp, making sure that multiple readings be taken at the exact same time but there can never be multiple of the same UUIDv1. so were the readingKey is I want the readingId to be teh one that gives the

```
1 409,
2 "A reading already exists for this branch and timestamp"
```

I also what the readingID and recordedAt to be split into to different columns

- Delete patch and put should also be RESTful APIs that the program should be able to uses ☒

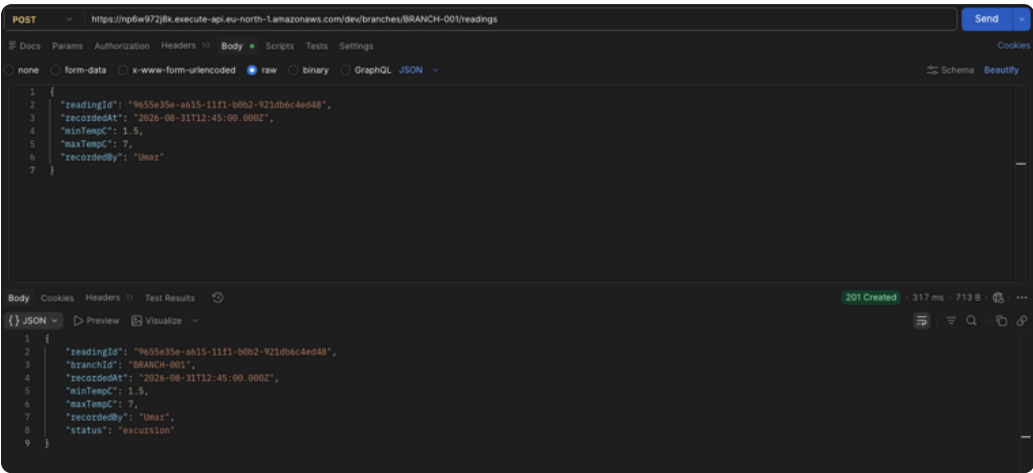
The patch doesn't have the ability to edit the whole reading. As the postman is imaging the client I don't think it should have full access to be able to change everything abut. reading only pessary thing they would need to change.

✅ can change	❌ cannot change
minTempC	readingId
maxTempC	branchId
recordedBy	recordedAt
😬	status
😬	alertRaisedAt

The DELET doesn't need a body of text it is just calling a reading via https:// and if it is successful it will output 204.

Then we did the tests POST, GET, PATCH and DELETE:

Post:



The image above shows the POST requests are working it creates a reading with the UUID separated into a different column to the time stamp. I created 5 of these as can be seeing teh image bellow.

Completed · Items returned: 5 · Items scanned: 5 · Efficiency: 100% · RCUs consumed: 2

Table: umar-coldchain-readings-terraform - Items returned (5)

Scan started on September 01, 2026, 16:01:13

☐	readingId (String)	alertRaisedAt	branchId	maxTempC	minTempC	recordedAt	recordedBy	status
☐	53820406-a615-11f1-b0b2-921db6c4ed48		BRANCH-001	7	3	2026-08-31...	Umar	ok
☐	9655e21e-a615-11f1-b0b2-921db6c4ed48		BRANCH-001	8	2	2026-08-31...	Umar	ok
☐	9655e4d0-a615-11f1-b0b2-921db6c4ed48		BRANCH-001	6	4	2026-08-31...	Umar	ok
☐	9655e35e-a615-11f1-b0b2-921db6c4ed48	2026-09-01T14:...	BRANCH-001	7	1.5	2026-08-31...	Umar	excursion
☐	9655e322-a615-11f1-b0b2-921db6c4ed48	2026-09-01T14:...	BRANCH-001	12	3	2026-08-31...	Umar	excursion

GET:

```
GET https://ng6w972jlk.execute-api.eu-north-1.amazonaws.com/dev/branches/BRANCH-001/readings

Body
200 OK · 1.55 s · 1.48 KB
JSON
1
{
  2
  {
  3    "maxTempC": 7,
  4    "recordedBy": "Umar",
  5    "alertRaisedAt": "2026-09-01T14:59:59.027Z",
  6    "status": "excursion",
  7    "readingId": "9655e35e-a615-11f1-b0b2-921db6c4ed48",
  8    "minTempC": 1.5,
  9    "branchId": "BRANCH-001",
 10    "recordedAt": "2026-08-31T12:45:00.000Z"
 11  },
 12
  {
 13    "minTempC": 3,
 14    "alertRaisedAt": "2026-09-01T14:59:47.987Z",
 15    "readingId": "9655e322-a615-11f1-b0b2-921db6c4ed48",
 16    "recordedBy": "Umar",
 17    "recordedAt": "2026-08-31T12:30:00.000Z",
 18    "status": "excursion",
 19    "maxTempC": 12,
 20    "branchId": "BRANCH-001"
 21  },
 22
  {
 23    "minTempC": 2,
 24    "recordedAt": "2026-08-31T12:15:00.000Z",
 25    "status": "ok",
 26    "recordedBy": "Umar",
 27    "readingId": "9655e21e-a615-11f1-b0b2-921db6c4ed48",
 28    "branchId": "BRANCH-001",
 29    "maxTempC": 8
 30  },
 31
  {
 32    "status": "ok",
 33    "maxTempC": 6,
```

The image above shows all readings from a single Branch being outputted.

```
GET https://ng6w972jlk.execute-api.eu-north-1.amazonaws.com/dev/branches/BRANCH-001/readings/9655e35e-a615-11f1-b0b2-921db6c4ed48

Body
200 OK · 662 ms · 751 B
JSON
1
{
  2    "alertRaisedAt": "2026-09-01T14:59:59.027Z",
  3    "branchId": "BRANCH-001",
  4    "maxTempC": 7,
  5    "minTempC": 1.5,
  6    "readingId": "9655e35e-a615-11f1-b0b2-921db6c4ed48",
  7    "recordedAt": "2026-08-31T12:45:00.000Z",
  8    "recordedBy": "Umar",
  9    "status": "excursion"
 10 }
```

Shows a single reading being outputted from a whole branch.

Patch:

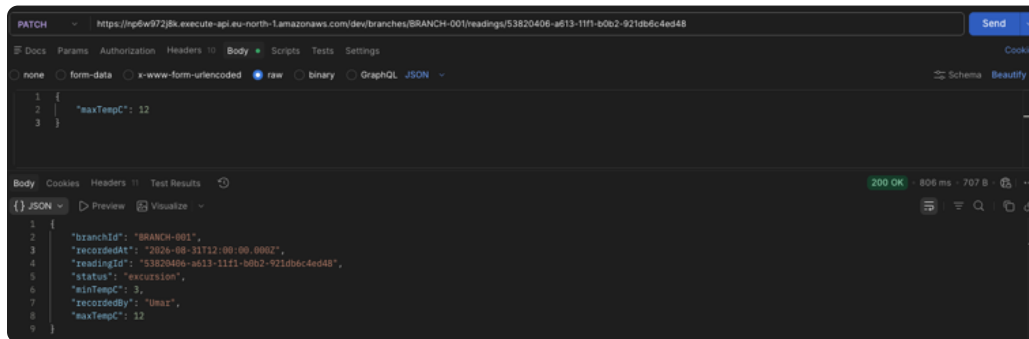
Completed · Items returned: 5 · Items scanned: 5 · Efficiency: 100% · RCUs consumed: 2

Table: umar-coldchain-readings-terraform - Items returned (5)

Scan started on September 01, 2026, 16:01:13

☐	readingId (String)	alertRaisedAt	branchId	maxTempC	minTempC	recordedAt	recordedBy	status
☐	53820406-a613-11f1-b0b2-921db6c4ed48		BRANCH-001	7	3	2026-08-31...	Umar	ok
☐	9655e21e-a613-11f1-b0b2-921db6c4ed48		BRANCH-001	8	2	2026-08-31...	Umar	ok
☐	9655de40-a613-11f1-b0b2-921db6c4ed48		BRANCH-001	6	4	2026-08-31...	Umar	ok
☐	9655e15e-a613-11f1-b0b2-921db6c4ed48	2026-09-01T14...	BRA...	7	1.5	2026-08-31...	Umar	excursion
☐	9655e122-a613-11f1-b0b2-921db6c4ed48	2026-09-01T14...	BRA...	12	3	2026-08-31...	Umar	excursion

The image above shows the database before the PATCH was ran. look at the top reading see how there is no excursion / alert value.



Once the PATCH was ran and max temp was changed to 12.

Completed · Items returned: 5 · Items scanned: 5 · Efficiency: 100% · RCUs consumed: 2

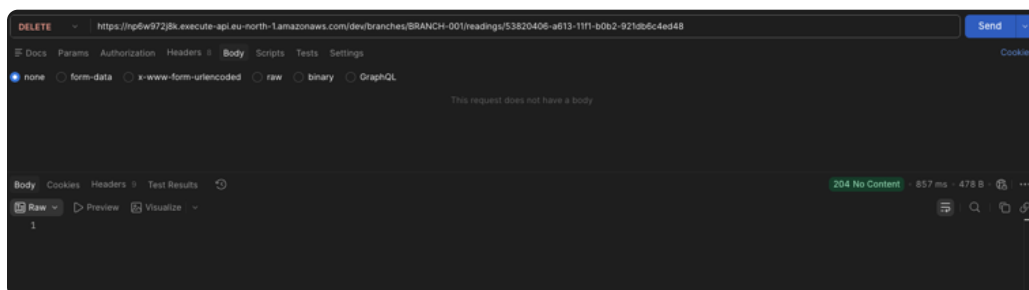
Table: umar-coldchain-readings-terraform - Items returned (5)

Scan started on September 01, 2026, 16:31:14

☐	readingId (String)	alertRaisedAt	branchId	maxTempC	minTempC	recordedAt	recordedBy	status
☐	53820406-a613-11f1-...	2026-09-01T15...	BRANCH-001	12	3	2026-08-31...	Umar	excursion
☐	9655e21e-a613-11f1-...		BRANCH-001	8	2	2026-08-31...	Umar	ok
☐	9655de40-a613-11f1-...		BRANCH-001	6	4	2026-08-31...	Umar	ok
☐	9655e15e-a613-11f1-...	2026-09-01T14...	BRANCH-001	7	1.5	2026-08-31...	Umar	excursion
☐	9655e122-a613-11f1-...	2026-09-01T14...	BRANCH-001	12	3	2026-08-31...	Umar	excursion

See how the excursion alert was raised and the value of max temp also changed.

Delete:



Then I ran the delete command and it delete the reading with the 204 No content.

Now we need to sort out:

- Whys the original database still on the terraform, should we remove it or will it mess up things what is the best course of action. ✓
- The image bellow show that if you want to create a reading you have to put in a UUID, I don't want this. because postman is supposed to be initiating the client and the client wouldn't have to input the UUID. I want the UUID to be randomly assigned when the reading is created and doesn't have to be inputted in the API request. ✓

```
1 {
2   "readingId": "53820406-a613-11f1-b0b2-921db6c4ed48",
3   "recordedAt": "2026-08-31T12:00:00.000Z",
4   "minTempC": 3,
5   "maxTempC": 7,
6   "recordedBy": "Umar"
7 }
```

Completed: now when a client inputs a reading all they need is the minTemp, maxTemp, and recordedAt values. they do not need the UUID making the log more realistic.

⚠ Questions / Queries / Pushback

Ask Luke what the final question in the learning objectives even means.

What does Terraform do about the order things get created in?